

Debug Console — comandos GDB (Blue Pill / STM32F103)

Referência rápida dos comandos úteis na **Debug Console** do VS Code durante uma sessão de debug com a extensão STM32Cube (adaptador `stlinkgdbtarget`, GDB `arm-none-eabi-gdb` do bundle). Complementa o fluxo de gravação/debug em [gravar-e-depurar.md](#).

Como a Debug Console interpreta o que você digita

A Debug Console tem **dois modos** de entrada:

Entrada	Interpretação	Exemplo
sem prefixo	avalia como expressão (no frame atual)	<code>uxTask, 2 + 3, *pxPort</code>
com prefixo <code>></code>	envia o comando cru ao GDB	<code>>show version, >info registers</code>

Por isso:

- **Comandos do GDB exigem o `>`**. Digitar `show version` sem o prefixo dá o erro `Evaluation of expression without frameId is not supported`, porque o VS Code tenta *avaliar uma expressão* e — se a CPU está rodando — não há frame de pilha selecionado. Com `>`, o GDB executa o comando independentemente de frame e o erro some.
- **Avaliar expressões** (nome de variável, `*ponteiro`, etc.) só funciona com o programa **parado (halted)** num breakpoint, pois aí existe um frame com escopo.

Confirmação de que o console está vivo (este ambiente):

```
>show version
GNU gdb (GNU Tools for STM32 14.3.rel1.20251027-0700) 15.2.90.20241229-git
...
This GDB was configured as "--host=x86_64-w64-mingw32 --target=arm-none-eabi".
```

Comandos GDB (use sempre com `>`)

Sessão / informação

Comando	O que faz
<code>>show version</code>	versão e configuração do GDB
<code>>help</code>	lista de categorias de ajuda
<code>>help <cmd></code>	ajuda de um comando específico (ex.: <code>>help break</code>)
<code>>info all-registers</code>	todos os registradores do core

Comando	O que faz
<code>>info registers</code>	registradores de uso geral (r0–r15, xPSR)

Execução

Comando	O que faz
<code>>continue / >c</code>	continua a execução (= F5)
<code>>next / >n</code>	step over (uma linha, sem entrar em funções)
<code>>step / >s</code>	step into (entra na função)
<code>>finish</code>	roda até retornar da função atual
<code>>interrupt</code>	pausa/halt o alvo (= botão Pause)

Os botões da barra de debug (F5/F10/F11) fazem o mesmo e são mais práticos. O console brilha mesmo é para o que **não tem botão** (abaixo).

Breakpoints / watchpoints

Comando	O que faz
<code>>break main</code>	breakpoint na função <code>main</code>
<code>>break main.c:42</code>	breakpoint no arquivo/linha
<code>>tbreak <local></code>	breakpoint temporário (some após o 1º hit)
<code>>info breakpoints</code>	lista breakpoints
<code>>delete <n></code>	remove o breakpoint nº <code>n</code>
<code>>watch uxCounter</code>	para quando a variável muda de valor
<code>>rwatch uxCounter</code>	para quando a variável é lida

Inspeção de memória / variáveis

Comando	O que faz
<code>>print uxCounter / >p uxCounter</code>	imprime valor (com <code>/x</code> para hex: <code>>p/x var</code>)
<code>>display uxCounter</code>	reimprime a cada parada
<code>>ptype <var></code>	mostra o tipo/struct
<code>>x/8xw 0x20000000</code>	dump de memória: 8 words em hex a partir do endereço
<code>>set var uxCounter = 0</code>	altera o valor de uma variável
<code>>backtrace / >bt</code>	pilha de chamadas do frame atual
<code>>frame <n> / >f <n></code>	seleciona um frame da pilha

Para **variáveis em escopo**, é mais simples digitá-las **sem** `>` (avaliação direta) ou usar o painel **Variables/Watch** do VS Code.

Comandos do servidor (`>monitor ...`)

`monitor` repassa o comando ao **ST-Link GDB Server** (camada OpenOCD-like). Úteis:

Comando	O que faz
<code>>monitor reset</code>	reseta o alvo
<code>>monitor halt</code>	para a CPU
<code>>monitor reg</code>	registradores via servidor

A disponibilidade exata depende do servidor; se um `monitor` não responder, use o equivalente puro do GDB (ex.: `>info registers` em vez de `>monitor reg`).

Periféricos (RCC, GPIO, ...) — use o viewer, não o console

Registradores de periféricos (ex.: `GPIOC->ODR`, `RCC->APB2ENR`) são melhor lidos pelo **viewer de periféricos** habilitado pelo `svdPath` no `launch.json` (extensão `stm32cube-ide-registers`), que decodifica os campos. Pelo console dá para ler o endereço cru, ex. o `ODR` do GPIOC (`0x4001100C`):

```
>x/1xw 0x4001100C
```

Dicas específicas de FreeRTOS

Com o kernel rodando (duas tasks + fila), o que costuma ajudar na hora de parar:

- `>bt` logo após um Pause mostra em qual task/contexto a CPU estava.
- `>print pxCurrentTCB` → TCB da task atual; `>print *pxCurrentTCB` expande a struct (nome da task em `pcTaskName` se `configMAX_TASK_NAME_LEN` permitir).
- `>print uxCurrentNumberOfTasks` → quantas tasks existem.
- Breakpoint dentro de uma task (`>break <func_da_task>`) para inspecionar a coordenação pela fila sem parar o scheduler manualmente.

Para visão de tasks/filas em árvore, o ideal é um plugin RTOS-aware do GDB; sem ele, os comandos acima cobrem o essencial.

Erro comum

Evaluation of expression without frameId is not supported → Você mandou um comando GDB **sem o** `>` (ou tentou avaliar uma expressão com a CPU rodando). Solução: use `>` para comandos GDB; para avaliar variáveis, pare num breakpoint primeiro.